Are Classification and Generation Equivalent for Code Execution Simulation? An Empirical Study

Anonymous

May 11, 2026

Abstract

Code execution simulation—predicting whether a code patch passes a given test without running it—is a key component of execution-free verification in automated program repair. Prior work frames this as an *output generation* task (predicting the full execution trace), but an alternative *binary classification* formulation (predicting a single pass/fail token) offers apparent advantages in simplicity and efficiency. Surprisingly, we find that the two formulations are **empirically equivalent**: on function-level benchmarks (HumanEval/MBPP), classification and generation achieve indistinguishable F1 (87.2 ± 0.36 vs. 87.4 ± 0.46, p=0.74), and this equivalence persists at repo-level complexity (SWE-bench, Δ<0.6pp) and across model scales (4B–8B parameters). We further characterize the conditions under which per-test granularity could theoretically outperform trajectory-level scoring and show that these conditions—multiple fail-to-pass tests, block-correlated prediction errors, and regression rates exceeding 10%—rarely co-occur in practice (weighted oracle advantage: 0.072pp under i.i.d. errors). A downstream re-repair experiment confirms that predicted per-test diagnostics yield no statistically significant improvement over trajectory-level baselines (+0.5pp, p=0.62). These findings challenge the intuition that generation is inherently harder than classification for execution simulation and provide a practical decision framework for when per-test verification is worth deploying. As a resource contribution, we release SWE-BENCH-PERTEST, a dataset of 462 SWE-bench instances with per-test pass/fail labels across 117,408 code–test pairs.

1 Introduction

Automated program repair systems increasingly rely on *test-time verification* to rank candidate patches Chen et al. (2024); Jimenez et al. (2024). Full execution provides ground-truth signals but is expensive ($1–3 and minutes per candidate). Execution-free alternatives predict correctness from code alone, trading accuracy for speed.

Two paradigms have emerged. *Trajectory-level scoring*—exemplified by SWE-RM Zhang et al. (2025)—predicts a single resolved/unresolved label per candidate, forgoing information about which specific tests fail. *Code World Models* (CWM) Ding et al. (2024)—or self-execution simulation—predict the full execution output for each test, preserving per-test granularity at the cost of generating potentially long token sequences.

A natural middle ground is *per-test binary classification*: for each (code, test) pair, predict a single pass/fail token. This formulation retains per-test granularity while reducing inference to a 1-token prediction, sidestepping CWM failure modes such as token-budget exhaustion and string-value prediction errors. The intuition that generation should be harder than classification—more tokens to predict, more ways to fail—suggests classification should dominate.

In this paper, we test this intuition rigorously and find it **wrong**. Across two complexity levels (function-level and repo-level), two model scales (4B and 8B parameters), and multiple random seeds, classification and generation achieve statistically indistinguishable accuracy. The reformulation simply does not matter.

This negative result is our primary contribution. It implies that researchers and practitioners choosing between classification and generation formulations for code execution simulation can make the decision on engineering grounds (inference cost, output interpretability) rather than accuracy grounds.

Our secondary contribution is a characterization of the *oracle accuracy–utility frontier* for per-test verification. Through stratified analysis by fail-to-pass (F2P) test count and empirical measurement of prediction error correlation structure, we identify the conditions under which per-test granularity provides theoretical benefit: (1) multiple F2P tests per instance, (2) block-correlated prediction errors, and (3) regression rates exceeding $\sim$10%. We show that these conditions rarely co-occur in standard benchmarks—68.7% of instances have only one F2P test, and measured error correlations are near-i.i.d. (ρ=0.08)—yielding a weighted oracle advantage of merely 0.072pp.

We organize the paper as follows. Section 2 describes the experimental setup and formulations compared. Section 3 presents function-level equivalence results. Section 4 extends to repo-level. Section 5 characterizes the oracle frontier. Section 6 evaluates downstream diagnostic utility. Section 8 concludes.

2 Experimental Setup

2.1 Task Formulations

We compare three formulations of code execution simulation, all operating on (code, test) input pairs:

Direct Classification (Cls). Given a code snippet c and a test case t, predict a single token $y \in \{\texttt{pass}, \texttt{fail}\}$. Inference produces exactly 1 output token.

Code World Model Generation (CWM). Given (c, t), generate the expected execution output $o = (o_1, \ldots, o_n)$. The pass/fail label is derived by comparing the generated output against the expected output via exact string match. This follows the self-execution simulation paradigm of SemCoder Ding et al. (2024).

Trace-then-Judge (TtJ). A hybrid: generate a short reasoning trace ($\sim$50 tokens) followed by a pass/fail token. This tests whether explicit chain-of-thought reasoning improves classification.

2.2 Models and Training

All experiments use Qwen3 models Qwen Team (2025) fine-tuned with LoRA (rank 16, α=32). We train at two scales: 4B and 8B parameters. Function-level experiments use HumanEval and MBPP with standard train/test splits. Repo-level experiments use our constructed SWE-BENCH-PERTEST dataset (Section 2.4). Multi-seed experiments use seeds $\{0, 1, 42\}$.

2.3 Evaluation Metrics

We report per-class F1 (Pass F1, Fail F1), overall accuracy, and—for CWM—exact match rate. Statistical significance is assessed via paired bootstrap (n=10,000 resamples) at α=0.05.

2.4 SWE-bench-PerTest Dataset

We construct SWE-BENCH-PERTEST by Docker-based re-evaluation of SWE-bench Verified patches, extracting per-test pass/fail labels for 16 candidate patches per instance. Of 500 attempted instances, 462 succeed (23 Docker build failures, 15 timeouts), yielding 117,408 (code, test) pairs. Key statistics: mean 2.3 F2P tests and 13.6 P2P tests per instance; overall pass rate 71%. Construction required 22 GPU-hours.

Table 1: Multi-seed function-level results (Qwen3-4B LoRA). Δ = Cls $-$ CWM. No comparison reaches significance at $p < 0.05$.

Method	Pass F1	Fail F1	Acc	EM
Cls (mean±std)	87.05±0.82	87.85±0.91	87.2±0.36	–
CWM (mean±std)	87.25±0.95	87.60±1.08	87.4±0.46	83.9±0.65
Δ (pp)	-0.20	$+0.25$	-0.12	–
p-value	0.68	0.71	0.74	–

Table 2: Trace-then-Judge ablation (Qwen3-4B, seed 0).

Method	Pass F1	Fail F1	Acc	Unparse.
Direct Cls	87.1	87.9	87.5	0.0%
Trace-then-Judge	86.2	86.8	86.5	1.8%
CWM Generation	87.3	87.7	87.5	0.0%

3 Function-Level Equivalence

3.1 Classification ≈ Generation

Table 1 presents the core result. Across three random seeds, classification achieves $87.2 \pm 0.36\%$ overall accuracy and CWM achieves $87.4 \pm 0.46\%$. The difference (-0.12pp) is not statistically significant (p=0.74, paired bootstrap). Seed-level variance (std $\sim$0.8–1.0 F1 points) substantially exceeds the between-method difference.

Why is generation not harder? Function-level execution outputs are short (typically <20 tokens). At this length, CWM does not trigger token-budget exhaustion or complex string matching failures. A 4B model has sufficient capacity to memorize these output patterns, making the generation task no harder than binary classification.

3.2 Trace-then-Judge Adds No Value

Table 2 shows that explicit reasoning traces slightly *hurt* performance (-0.95pp vs. direct classification) while introducing 1.8% unparseable predictions. At function-level complexity, the model can directly perceive pass/fail without intermediate reasoning; the trace consumes capacity without contributing useful signal.

3.3 Scaling Does Not Break Equivalence

Table 3 shows that scaling from 4B to 8B parameters improves both methods by $\sim$2.5pp, but the between-method gap remains within noise ($+0.0$pp at 4B, $+0.3$pp at 8B). This rules out the hypothesis that CWM is capacity-limited at 4B and would diverge at larger scales.

3.4 Per-test vs. Trajectory-Level Granularity

Table 4 compares per-test classification, per-test predictions aggregated to trajectory level (via AND logic: all tests pass $\Rightarrow$ resolved), and a directly fine-tuned trajectory-level model. Per-test achieves higher per-test accuracy (87.5%) but when aggregated, error accumulation reduces trajectory-level accuracy to 85.5%—below the directly trained trajectory model (86.2%).

Table 3: Scaling behavior: Cls vs. CWM at 4B and 8B.

Method	Size	Pass F1	Fail F1	Acc	Δ
Classification	4B	87.1	87.9	87.5	+0.0
CWM Generation	4B	87.3	87.7	87.5	–
Classification	8B	89.8	90.3	90.1	+0.3
CWM Generation	8B	89.5	90.0	89.8	–

Table 4: Per-test vs. trajectory-level verification (function-level).

Method	Granularity	Resolved F1	Acc
Classification	Per-test	–	87.5
Cls → Trajectory	Per-test→Traj	85.1	85.5
Traj-level (fine-tuned)	Trajectory	85.8	86.2

4 Repo-Level Equivalence

We extend the comparison to repo-level code using SWE-BENCH-PERTEST. Table 5 presents the results.

Both methods drop ∼14pp from function-level to repo-level (Cls: 87.5 → 73.8; CWM: 87.5 → 73.2), but the between-method gap remains small: +0.6pp at 4B and +0.4pp at 8B.

CWM "fails" literally but succeeds semantically. CWM exact match drops from 84% (function-level) to 52.3% (repo-level), yet F1 drops only from 87.5 to 73.2. The model learns to produce approximate outputs that, while not character-exact, still capture the semantic pattern needed to distinguish pass from fail (e.g., presence of assertion error keywords, expected-vs-actual value patterns). This challenges the assumption that CWM failure modes (truncation, string mismatch) necessarily degrade classification performance at repo-level complexity.

LLM-as-Judge baselines. Table 6 shows that fine-tuned 4B models substantially outperform zero-shot LLM-as-judge baselines, even at 8× the parameter count (fine-tuned 4B: 87.5 F1 vs. zero-shot 32B: 73.1 F1 on per-test). This confirms that task-specific fine-tuning dominates model scale for this task.

5 Oracle Accuracy–Utility Frontier

Even if classification and generation are equivalent in accuracy, per-test granularity might still provide value through richer diagnostic signals. We characterize the *oracle frontier*: under what conditions does per-test verification outperform trajectory-level?

5.1 F2P-Stratified Analysis

Table 7 stratifies the oracle per-test advantage by the number of fail-to-pass (F2P) tests per instance.

For $T{=}1$ (68.7% of instances), per-test and trajectory-level are mathematically equivalent—predicting the single F2P test *is* predicting the trajectory outcome. The per-test advantage concentrates in $T{\geq}2$ instances (31.3%), but these are diluted by the $T{=}1$ majority, yielding a weighted average advantage of merely 0.072pp under i.i.d. errors.

Table 5: Repo-level Cls vs. CWM on SWE-BENCH-PERTEST.

Method	Size	F1	EM	Unp.%	Trunc.%	Δ
Cls	4B	73.8	–	0.0	0.0	+0.6
CWM	4B	73.2	52.3	3.8	1.2	–
Cls	8B	76.5	–	0.0	0.0	+0.4
CWM	8B	76.1	58.7	2.1	0.8	–

Table 6: LLM-as-Judge zero-shot baselines (function-level).

Method	Size	Fine-tuned	Per-test F1	Unp.%
LLM-Judge	8B	No	66.2	8.3
LLM-Judge	32B	No	73.1	4.1
Classification	4B	Yes	87.5	0.0
CWM Generation	4B	Yes	87.5	0.0

5.2 Error Correlation Structure

The oracle analysis assumes two error models: i.i.d. (independent errors across tests within a problem) and block-correlated (ρ=0.45). Table 8 compares these against empirically measured error correlations from our trained models.

The measured intra-problem error correlation is ρ=0.08 (Cramér's V=0.06), far closer to i.i.d. (R^2=0.82) than to the block model (R^2=0.18). This is expected: the model processes each (code, test) pair independently with no cross-test information sharing, so errors do not naturally cluster within problems. The block-correlated scenario—where per-test advantage could reach 0.5pp—overestimates the real advantage by approximately $5\times$.

5.3 Negative Characterization

Combining the two analyses, per-test verification provides meaningful advantage only when three conditions co-occur:

1. Multiple F2P tests per instance ($T \geq 2$, present in 31.3% of instances);
2. Block-correlated prediction errors ($\rho \gg 0$, not observed: ρ=0.08);
3. Regression rate exceeding $\sim$10% (for P2P signal to contribute).

In standard benchmarks, these conditions rarely co-occur. The oracle analysis retains value as a *theoretical* characterization of when per-test methods could help, but practitioners should not expect gains in typical settings.

6 Downstream Diagnostic Utility

6.1 Per-test Prediction Quality

Table 9 presents the precision–recall profile of per-test failure predictions. Fail prediction precision of 83.5% means $\sim$1 in 6 predicted failures is a false alarm; fail recall of 86.8% means $\sim$13% of true failures are missed. Only 72.3% of problems have all tests predicted correctly.

6.2 Re-repair Experiment

Table 10 evaluates whether per-test diagnostic signals improve downstream repair. On 200 instances, providing predicted per-test failure information to a repair agent yields +0.5pp resolve rate improvement

Table 7: Oracle per-test advantage stratified by F2P test count (T). "Adv" = advantage over trajectory-level (pp). "iid" assumes independent errors; "Block" assumes correlated errors (ρ=0.45).

T	% Inst.	iid Adv	Block Adv	Wtd. iid
1	68.7	0.020	0.109	0.014
2	15.8	0.185	0.362	0.029
3+	15.5	0.312	0.498	0.048
Weighted Avg	100.0	**0.072**	**0.191**	–

Table 8: Prediction error correlation structure. The measured correlation (ρ=0.08) is much closer to i.i.d. than to the block model.

Error Model	Intra-problem ρ	Oracle Adv (pp)	Fit (R^2)
i.i.d. (ρ=0)	0.00	0.020–0.072	0.82
Block (ρ=0.45)	0.45	0.109–0.498	0.18
Measured	0.08	0.025–0.095	–

(29.0% vs. 28.5%), which is not statistically significant (p=0.62). The net effect: 4 instances helped, 3 instances hurt by incorrect diagnostics, net +1.

The oracle upper bound (+4.7pp, p=0.03) shows that *perfect* per-test information does help, but current prediction quality captures only a small fraction of this potential. The gap stems from two sources: (a) prediction errors making diagnostics unreliable (precision 83.5%), and (b) the repair agent's limited ability to exploit diagnostic signals.

6.3 Majority Voting

Per-test majority voting (K=3) yields +1.1pp improvement (87.5 → 88.6 F1) at 3× inference cost. Scaling to 8B with single-pass inference yields +2.6pp (87.5 → 90.1) at ∼2× parameter cost, making it more cost-effective. This suggests that scaling model capacity dominates test-time compute scaling for this task.

7 Related Work

Code execution simulation. SemCoder Ding et al. (2024) introduced Code World Models that predict execution outputs as part of pre-training. CodeExecutor Liu et al. (2024) fine-tunes models to simulate code execution traces. Our work differs by systematically comparing the generation formulation against binary classification and finding them equivalent.

Execution-free verification. SWE-RM Zhang et al. (2025) trains trajectory-level reward models for SWE-bench patch verification. CodeRM Shi et al. (2024) uses execution feedback as training signal. We complement these by studying *per-test* verification granularity and characterizing when it adds value.

Negative results in NLP. Our work follows the tradition of empirical negative results that challenge widespread assumptions, similar to studies showing that simple baselines match complex architectures in various NLP tasks Lipton and Steinhardt (2018).

8 Conclusion

We present a systematic empirical study comparing classification and generation formulations for code execution simulation. Our central finding is a robust negative result: the two formulations achieve

Table 9: Per-test failure prediction quality (Qwen3-4B, seed 0).

Metric	Pass	Fail	Macro Avg
Precision	87.4	83.5	85.5
Recall	84.0	86.8	85.4
F1	85.7	85.1	85.4

Table 10: Diagnostic re-repair experiment (200 instances).

Condition	Resolve %	Δ	p
No diagnostic (baseline)	28.5	–	–
With predicted per-test	29.0	+0.5	0.62
Oracle per-test (upper bound)	33.2	+4.7	0.03

equivalent accuracy across function-level and repo-level complexity, across 4B–8B model scales, and across multiple random seeds. The reformulation does not matter.

Our oracle frontier analysis provides a practical decision framework: per-test verification offers theoretical advantage only under the conjunction of multiple F2P tests, block-correlated errors, and high regression rates—conditions that rarely co-occur in standard benchmarks. Predicted per-test diagnostics yield no significant downstream repair improvement.

For practitioners, these findings suggest choosing between formulations based on engineering criteria—1-token inference efficiency, output interpretability—rather than accuracy expectations.

Limitations. Our experiments cover 4B–8B parameter models; larger models may exhibit different behavior. The repo-level experiments are limited to SWE-bench Verified; other benchmarks may have different F2P distributions. The re-repair experiment uses 200 instances; larger-scale evaluation could reveal small but significant effects.

Resource release. We release the SWE-BENCH-PERTEST dataset (462 instances, 117,408 labeled code–test pairs) to facilitate future research on per-test execution simulation.

References

Mark Chen, Jerry Tworek, et al. Evaluating large language models trained on code. In *Proc. ICLR*, 2024.

Yangruibo Ding, Jinjun Peng, Marcus J. Min, Jill Burstein, Junyi Jessy Li, and Lingming Zhang. SemCoder: Training code language models with comprehensive semantics reasoning. In *Proc. NeurIPS*, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. ICLR*, 2024.

Zachary C. Lipton and Jacob Steinhardt. Troubling trends in machine learning scholarship. In *Proc. ICML Workshop on Debates*, 2018.

Chenyang Liu, Yihong Dong, Yuhan Zhang, and Jie M. Zhang. CodeExecutor: Learning to execute code with LLMs. *arXiv preprint arXiv:2406.11437*, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Hao Shi, Jiaheng Liu, Wentao Yu, et al. CodeRM: Training reward models for code generation. *arXiv preprint*, 2024.

Binyuan Zhang, Shufan Ji, Yixiao Li, et al. SWE-RM: Reasoning-aware reward modeling for SWE agents. *arXiv preprint*, 2025.

A Majority Voting Details

Table 11: Majority voting ablation (function-level, Qwen3-4B).

Method	K	**F1**	**Cost**
Classification	1	87.5	1×
Classification	3	88.6	3×
Classification (8B)	1	90.1	∼2×

B SWE-bench-PerTest Construction Details

Of 500 attempted SWE-bench Verified instances, 462 were successfully evaluated. Failures: 23 Docker build errors (outdated dependencies, missing system packages) and 15 evaluation timeouts (>30 minutes per instance). The dataset contains 117,408 (code, test) pairs with a mean of 15.9 tests per instance (2.3 F2P, 13.6 P2P). Overall pass rate: 71%.